CCA 3 — COMPUTER VISION PRACTICAL ASSIGNMENT

Official 6–8 Minute Video Presentation & Spoken Dialogue Script

Student Name: Yash Ranpariya	**PRN:** 272251066	**Institution:** MIT-WPU Pune
Course: MCA (Term III)	**Faculty:** Prof. Rahul Inchal	**GitHub:** github.com/Yash-Ranpariya/cv-

[0:00 – 1:00] Section 1: Introduction & Dataset Files

ON SCREEN: Open **main.py** in VS Code and show the **dataset/** folder with original_photo.jpg, second_photo.jpg, and original_video.mp4.

SPOKEN WORDS:

Hello, respected faculty and evaluators. My name is **Yash Ranpariya**, PRN **272251066**, studying Master of Computer Applications at MIT World Peace University, Pune. Today, I am presenting my Continuous Assessment 3 practical coding assignment in Computer Vision, covering Units 3, 4, and 5, under the guidance of Prof. Rahul Inchal.

For this assignment, I captured my own authentic dataset without using stock photos: First, **original_photo.jpg** for restoration, feature extraction, and recognition; Second, **second_photo.jpg** from an alternate perspective for feature matching; and Third, **original_video.mp4**, a self-recorded clip for motion estimation. Let us now walk through the implementation and results for each task.

[1:00 – 2:30] Section 2: Task 1 — Image Restoration & Segmentation (Unit 3)

ON SCREEN: Open **outputs/task1_restoration_segmentation/fig4_comprehensive_restoration_grid.png** and **fig8_watershed_segmentation_pipeline.png**.

SPOKEN WORDS:

Task 1 addresses two problems: image restoration and semantic object segmentation.

Part A: Image Restoration: I degraded my photo with two noise models: Additive White Gaussian Noise (AWGN, sigma=25) simulating electronic sensor noise, and Salt-and-Pepper impulse noise (3%) simulating transmission bit errors.
I applied both a linear 2D Gaussian filter and a non-linear Median filter. On Gaussian noise, Gaussian filtering achieved **28.82 dB PSNR and 0.7405 SSIM**. However, on Salt-and-Pepper noise, Gaussian filtering smeared impulses into cloudy patches (26.67 dB). In contrast, the **Median filter achieved an outstanding 31.31 dB PSNR and 0.8949 SSIM** because sorting the neighborhood completely rejects extreme 0 and 255 outliers, keeping edges sharp.

Part B: Morphological Operations & Segmentation: I applied Otsu's thresholding to maximize between-class variance, followed by morphological erosion, dilation, opening, and closing. To segment touching regions, I computed the Euclidean Distance Transform to extract sure foreground seeds and executed marker-controlled **Watershed Segmentation**, highlighted by the red boundary lines.

[2:30 – 4:00] Section 3: Task 2 — Feature Extraction & Invariant Matching (Unit 4)

ON SCREEN:	Open **outputs/task2_feature_extraction/fig1_hog_visualization.png** and **fig3_matching_comparison_summary.png**.

SPOKEN WORDS:	Task 2 investigates local gradient and scale-invariant descriptors. **Histogram of Oriented Gradients (HOG):** HOG computes 9-bin gradient histograms across normalized 8x8 pixel cells. It extracted **489,168 features** from my photo, capturing prominent object contours visible in the inferno visualization. **Scale-Invariant Feature Transform (SIFT):** SIFT detects stable extrema in Difference-of-Gaussians scale space, assigns a dominant orientation, and builds a 128-dimensional descriptor. I detected 2,000 keypoints per photo and matched them using Lowe's Ratio Test (0.75 cutoff) and RANSAC homography. On the original scene pair, SIFT achieved **220 good matches with 111 RANSAC inliers (50.5% inlier ratio)**. To test invariance, I rotated Photo 2 by **35 degrees and scaled it to 0.8x**. SIFT maintained **219 matches and 106 inliers (48.4% inlier ratio)**, experimentally proving scale and rotational invariance.

[4:00 – 5:30] Section 4: Task 3 — Object Recognition (Viola–Jones vs YOLOv8) (Unit 5)

ON SCREEN:	Open **outputs/task3_object_recognition/fig1_detection_comparison_montage.png**.

SPOKEN WORDS:	Task 3 contrasts classical geometric detection against deep convolutional neural networks. **Viola–Jones Haar Cascade:** Uses rectangular Haar features accelerated by an Integral Image and selected via AdaBoost in an attentional cascade. It successfully detected **2 frontal faces** (blue boxes). However, it is restricted to frontal human faces with no continuous confidence score, failing when head tilt exceeds 20 degrees. **Ultralytics YOLOv8:** An anchor-free deep single-stage CNN with a CSPDarknet backbone and PANet pyramid. It detected **3 objects** across MS COCO classes with calibrated continuous probability scores and high angle tolerance. This highlights the evolution from rigid handcrafted heuristics to generalized hierarchical representation learning.

[5:30 – 7:00] Section 5: Task 4 — Video Motion Estimation (Unit 5)

ON SCREEN:	Open **outputs/task4_motion_estimation/fig1_motion_estimation_comparison_grid.png** and **08_lucas_kanade_motion_vectors.png**.
	Task 4 analyzes temporal dynamics in my video using four complementary algorithms: 1. **Temporal Frame Differencing:** Subtracted adjacent frames to detect **6 motion clusters**. Very fast, but subject to ghosting when motion pauses. 2. **GMM Background Subtraction (MOG2):** Models each background pixel with 3 to 5 Gaussians, isolating moving silhouettes while filtering dynamic shadows. 3. **Lucas–Kanade Sparse Optical Flow:** Tracks Shi-Tomasi corners under the brightness constancy assumption. It computed **215 motion vectors with an average speed of 2.64 px/frame**, rendered as yellow arrows indicating travel direction. 4. **Dense Farneback Flow:** Computes pixel-wise flow visualized via an HSV color wheel (hue represents direction, brightness represents velocity).
SPOKEN WORDS:	While frame differencing and GMM provide spatial presence masks, optical flow delivers true velocity vectors essential for tracking.

[7:00 – 7:30] Section 6: Conclusion & Repository Verification

ON SCREEN:	Show terminal with **python main.py --validate** PASS status, or the GitHub repository at **github.com/Yash-Ranpariya/cv-**.
	In conclusion, this project implemented and verified the entire computer vision continuum: restoration, segmentation, invariant feature extraction, object recognition, and multi-paradigm video motion estimation. All source code is committed to my GitHub repository at **github.com/Yash-Ranpariya/cv-**, and the original video recording has been shared on Google Drive with Prof. Rahul Inchal.
SPOKEN WORDS:	Thank you very much for your time and guidance.